Table 8.1 Common callback handlers in LangChain.js

Event	Trigger	Method
Chain start	When a chain starts running	`handleChainStart`
Chain end	When a chain ends	`handleChainEnd`
Chain error	When a chain errors	`handleChainError`
LLM start	When an LLM starts	`handleLlmStart`
LLM new token	When an LLM or chat model emits a new token	`handleLlmNewToken`
LLM end	When an LLM or chat model ends	`handleLlmEnd`
LLM errors	When an LLM or chat model encounters an error	`handleLlmError`

Callback handlers in LangChain.js can be injected not only within the LLM provider calls but also within individual invoke-method calls, providing a more fine-grained tracing capability. Here is an example:

```
console.log(await finalChain.invoke(
{ question: 'What is artificial intelligence?' },
{callbacks: [handler]}));
```

All available options to capture traces and logs from your chain execution processes are extremely helpful for debugging purposes, and you should utilize them in full when working with LangChain.js.

8.3.2 Troubleshooting model integration problems

Model integration problems typically arise when setting up and configuring language models within LangChain.js. These problems can lead to unexpected behaviors, errors, and inefficiencies in how models interact with other components of the system. There are many situations when LangChain.js chains will fail to run effectively, so be sure to set up a reliable logging system to capture any errors.

VECTOR DIMENSIONS MISMATCH

This problem arises when the dimensions of the embeddings produced by a model do not match the expected dimensions in the vector store. For example, if a vector store is configured for 1,024-dimensional embeddings but receives 8,192-dimensional vectors, it will throw an error. This mismatch can arise from two reasons:

- *Incompatible models*—Using a model that produces embeddings with dimensions different than those expected by the vector store can lead to integration failures.
- *Incorrect configuration*—Misconfigurations in either the embedding model or the vector store setup can result in dimension mismatches.

We can reproduce vector dimension mismatch by reviewing the code we discussed in the previous chapter when parsing the PDF documents into the HNSWLib vector store.